Nombre:_____ Matrícula:_____

Instrucción
Este examen es un proyecto individual de una semana en el que construirás, de principio a fin, una aplicación de consola en Java (aplicando los principios del Diseño y POO): la analizas, la diseñas en UML, la programas y la publicas en GitHub con documentación profesional.
Nota. Lo importante es que puedas explicar por qué diseñaste cada clase y cada relación. La IA puede ayudarte a resolver dudas, pero el modelo, el código y el README deben reflejar lo que tú entendiste.

Caso de estudio: clínica veterinaria PetCare

La clínica veterinaria PetCare necesita un sistema de consola para registrar a sus mascotas y dueños, agendar citas, registrar consultas y calcular lo que cobra por cada servicio. Tú eres el equipo de desarrollo: debes entender el problema, modelarlo y construirlo.

Descripción del problema

- **Personas**. En la clínica hay dueños, veterinarios y recepcionistas. Todos tienen nombre, apellido, teléfono y correo. Cada tipo tiene datos propios: el dueño tiene dirección, el veterinario tiene cédula profesional y especialidad, y el recepcionista tiene turno.
- **Mascotas**. Cada mascota tiene nombre, edad, peso y un dueño. Hay al menos tres tipos: perro, gato y ave exótica. Cada tipo tiene un dato propio (por ejemplo, raza y nivel de energía del perro; si el gato es_domestico; la longitud_alas del ave) y emite un sonido distinto.
- **Historial clínico**. Cada mascota tiene un historial clínico donde se guardan sus consultas. El historial no tiene sentido sin la mascota: si la mascota se elimina del sistema, su historial también.
- **Consultorios**. La clínica tiene consultorios numerados. Los consultorios pertenecen a la clínica y desaparecen con ella.
- **Personal**. La clínica cuenta con un equipo de veterinarios. Un veterinario existe aunque la clínica cierre y puede trabajar en otra.
- **Citas**. Una cita relaciona a una mascota con un veterinario en una fecha y hora, y ocurre en un consultorio. Una cita puede estar programada, atendida o cancelada.
- **Servicios y cobro**. Los servicios son consulta general, vacunación y cirugía. Cada uno calcula su costo de forma distinta: la consulta tiene tarifa base, la vacunación suma el costo de la vacuna y la cirugía depende de la complejidad y de las horas de quirófano. Al terminar una cita se genera un comprobante con el total.

Reglas de negocio

1. El peso y la edad de una mascota no pueden ser negativos ni cero.
2. Un veterinario no puede tener dos citas programadas a la misma hora.
3. Una cita cancelada no puede marcarse como atendida.
4. Un consultorio no puede usarse en dos citas al mismo tiempo.
5. Los costos nunca pueden ser negativos.
6. Todo cobro debe poder consultarse por mascota y por veterinario.

Alcance

No se pide base de datos, interfaz gráfica ni conexión a internet. Los datos viven en memoria mientras el programa corre, usando colecciones de Java. Si quieres guardar en archivos es un extra opcional.

Parte 1: análisis y diseño con UML

Antes de escribir código, modela la solución en una herramienta UML (por ejemplo draw.io, StarUML, Lucidchart, PlantUML o Visual Paradigm). Exporta cada diagrama como imagen PNG o PDF y guárdalo en el repositorio.

1.1 Identificación de clases y objetos

A partir del caso de estudio, elabora una tabla de análisis con este formato (mínimo 12 clases):

Clase candidata	Tipo (concreta, abstracta o interfaz)	Atributos principales	Responsabilidades (métodos)	Justificación

Incluye también tres objetos de ejemplo con sus valores, y explica en una línea por qué descartaste al menos dos clases candidatas que no convenía modelar.

1.2 Diagrama de clases

El diagrama debe mostrar:

- Atributos con **visibilidad** (`+`, `-`, `#`, `~`) y tipo.
- Métodos con visibilidad, parámetros y tipo de retorno, incluidos constructores y métodos sobrecargados.
- Al menos una **herencia** con clase abstracta (nombre en cursiva o estereotipo `<<abstract>>`).
- Al menos una segunda **jerarquía de herencia**, distinta de la anterior.
- Al menos una **interfaz** con dos o más clases que la implementan.
- Al menos una **asociación** con multiplicidad y nombre del rol.
- Al menos una **agregación** y una **composición**, con la notación correcta.
- La organización en **paquetes** (subpaquetes del diagrama).

1.3 Justificación de relaciones

Para cada relación de tipo asociación, agregación y composición, escribe una o dos frases que expliquen por qué elegiste ese tipo y no otro. Piensa en el ciclo de vida: ¿**qué pasa con una de las partes si la otra se elimina?**

1.4 Diagrama de casos de uso (opcional pero valorado)

Identifica los actores (recepcionista, veterinario, dueño) y al menos seis casos de uso del sistema.

Parte 2: implementación en Java

Implementa en Java (versión 17 o superior) el diseño que hiciste en la Parte 1. El código debe coincidir con tu diagrama; si cambias el diseño al programar, actualiza el diagrama.

Tema	Requisito mínimo en el código
Definición de clases	Mínimo 12 clases con atributos y métodos bien definidos, cada una con una sola responsabilidad clara.
Ámbito de atributos y métodos	Uso justificado de variables de instancia, variables locales y miembros static (por ejemplo, un contador de comprobantes generados).
Creación y eliminación de objetos	Creación con new y constructores. Elimina objetos del sistema (por ejemplo, dar de baja una mascota con su historial) y explica en comentarios qué ocurre con los objetos sin referencias (recolector de basura).
Constructores	En cada clase concreta, al menos un constructor con parámetros. En las clases hijas, uso de super(...). En al menos una clase, un constructor por defecto y otro que reciba todos los datos.
Sobrecarga	Al menos dos casos de sobrecarga de métodos o constructores con firmas distintas (por ejemplo, buscarCita(String) y buscarCita(LocalDate)).
Encapsulamiento	Todos los atributos private o protected según corresponda. Getters y setters solo donde tengan sentido; los setters validan las reglas de negocio y lanzan IllegalArgumentException cuando el dato no es válido.
Herencia	Dos jerarquías: Persona y Mascota, con atributos y comportamientos comunes en la clase padre.
Clases abstractas	Al menos una clase abstracta (por ejemplo, Persona o Mascota) con un método abstracto implementado en cada hija.
Sobreescritura	Al menos tres métodos sobreescritos con @Override, incluido toString() en las clases principales.
Asociación	Una relación entre objetos independientes, por ejemplo Cita que referencia a Mascota, Veterinario y Consultorio.
Agregación	La clínica agrupa veterinarios que existen por sí mismos: se reciben ya creados y siguen existiendo si la clínica se elimina.
Composición	La clínica crea y posee sus consultorios, y la mascota crea y posee su historial clínico. Las partes se crean dentro del objeto contenedor y no se comparten.
Paquetes	Código organizado en al menos cuatro paquetes (por ejemplo modelo.personas, modelo.mascotas, modelo.servicios, app), con importaciones explícitas.
Niveles de acceso	Uso intencional de private, protected, el acceso por defecto (de paquete) y public, con un comentario breve en dos lugares donde expliques por qué elegiste ese nivel.
Interfaces	Al menos una interfaz (por ejemplo Facturable con calcularCosto()) implementada por tres o más clases (consulta, vacunación, cirugía).

Tema	Requisito mínimo en el código
Polimorfismo	Una lista de tipo List<Mascota> que contiene perros, gatos y aves y llama a hacerSonido(), y una lista List<Facturable> que calcula el costo total sin preguntar el tipo con instanceof.

Menú de consola

La aplicación debe ejecutarse desde una clase Main con un menú que se repite hasta que el usuario elige salir. Debe incluir como mínimo estas opciones:

```
1. Registrar dueño y mascota (perro, gato o ave).
2. Registrar veterinario y asignarlo a la clínica.
3. Agendar una cita.
4. Atender o cancelar una cita.
5. Generar el comprobante de una cita atendida.
6. Consultar el historial clínico de una mascota.
7. Listar todas las mascotas y su sonido (demostración de
   polimorfismo).
8. Ver el total cobrado por veterinario.
9. Dar de baja una mascota.
10. Salir.
```

El programa debe validar entradas incorrectas (texto donde se espera un número, opciones fuera del menú, datos vacíos) sin cerrarse de forma abrupta.

Escenario de demostración obligatorio

Incluye en el README una captura de pantalla o un bloque de texto con la ejecución de este escenario:

1. Registrar dos dueños y tres mascotas (un perro, un gato y un ave).
2. Registrar dos veterinarios.
3. Agendar tres citas y atender dos, de modo que se calculen los costos de consulta, vacunación y cirugía.
4. Intentar agendar una cita que viole una regla de negocio y mostrar el mensaje de error.
5. Listar las mascotas llamando a hacerSonido() para mostrar el polimorfismo.
6. Mostrar el total cobrado y consultar un historial clínico.

Convenciones de codificación

Sigue las convenciones oficiales de Java. Se revisarán en toda la entrega.

Elemento	Convención	Ejemplo
Clases e interfaces	UpperCamelCase, sustantivos	`Veterinario`, `Facturable`
Métodos	lowerCamelCase, verbos	`agendarCita()`, `calcularCosto()`
Atributos, variables	lowerCamelCase, nombres que describen su contenido	`fechaNacimiento`
Constantes	MAYUSCULAS_CON_GUION_BAJO	`TARIFA_BASE`
Paquetes	minúsculas, sin guiones ni acentos	`modelo.mascotas`
Archivos	Una clase pública por archivo, con el mismo nombre de la clase	`Perro.java`
Sangría y llaves	4 espacios, sin tabuladores; llave de apertura en la misma línea	`if (peso <= 0){`

Elemento	Convención	Ejemplo
Comentarios	Javadoc (/** ... */) en cada clase y método público; comentarios internos solo donde aclaren una decisión	

Además:

- Escribe nombres de clases, métodos y atributos en español o en inglés, pero no mezcles idiomas dentro del proyecto.
- Evita métodos de más de 30 líneas y líneas de más de 100 caracteres.
- No dejes código comentado, variables sin usar ni `System.out.println` de depuración.
- Cada commit y cada archivo deben estar libres de errores de compilación.

Parte 3: control de versiones con Git y GitHub

Tu repositorio es la evidencia de cómo construiste la solución, no solo el resultado final. Se revisará el historial de commits.

1. Crea un repositorio público en GitHub con un nombre descriptivo, por ejemplo `petcare-poo-java`.
2. Haz commits pequeños y frecuentes durante toda la semana, no uno solo al final. Se esperan al menos 10 commits distribuidos en al menos cuatro días distintos.
3. Escribe mensajes de commit claros, en imperativo y en un solo idioma. Ejemplo: `Agrega clase abstracta *Mascota* con validación de *peso*`.
4. Usa una rama *main* estable y al menos dos ramas de trabajo (por ejemplo *feature/herencia-mascotas* y *feature/citas*), que fusionas con merge o pull request.
5. Publica al menos una versión (release o tag) llamada *v1.0.0* con la entrega final.
6. Incluye un archivo *.gitignore* para Java, de modo que no subas archivos compilados (*.class*), carpetas *out/* o *bin/*, ni archivos del IDE.
7. No subas contraseñas, rutas personales ni datos privados.

README.md profesional

El README es la portada de tu proyecto: quien lo lea debe entender qué hace, cómo ejecutarlo y cómo está diseñado sin abrir el código. Escríbelo en Markdown, con tus propias palabras.

Secciones obligatorias

Sección	Qué debe contener
Título y descripción	Nombre del proyecto y un párrafo que explique qué problema resuelve.
Tecnologías	Versión de Java, herramienta UML e IDE utilizados.
Modelo de la solución	El diagrama de clases (imagen incrustada) y una explicación breve de las decisiones de diseño: qué es herencia, qué es interfaz y por qué elegiste agregación o composición en cada caso.
Estructura del proyecto	Árbol de carpetas y paquetes, con una línea que explique el propósito de cada paquete.
Conceptos aplicados	Una tabla que indique en qué archivo y línea se aplica cada tema: herencia, clase abstracta, interfaz, polimorfismo, sobrecarga, composición, etc.

Sección	Qué debe contener
Instalación y ejecución	Pasos exactos para clonar, compilar y ejecutar desde la consola. Alguien que no conoce tu proyecto debe poder hacerlo sin ayuda.
Ejemplo de uso	Capturas o bloques de texto con el escenario de demostración y su salida.
Reglas de negocio	Cómo y dónde se validan en el código.
Reflexión	Qué aprendiste, qué fue lo más difícil, qué mejorarías y cómo usaste la IA, si la usaste.
Autor	Tu nombre y tu perfil de GitHub.

Ejemplos para observar en GitHub

Revisa estos repositorios antes de escribir el tuyo. No son proyectos de Java ni de este tema; sirven para ver cómo se organiza un README claro y atractivo.

- awesome-readme: una lista curada de READMEs destacados, con una nota sobre qué hace bien cada uno (logo, capturas, tabla de contenido, instrucciones de instalación, ejemplos de uso).
- Best-README-Template: una plantilla muy usada con secciones como descripción del proyecto, primeros pasos, uso y contacto. Úsala como inspiración, no la copies tal cual.

Te conviene fijarte en cómo cada uno responde tres preguntas en los primeros segundos: qué es, cómo se instala y cómo se usa.

Buenas prácticas de redacción

- Usa encabezados (#, ##) para estructurar el documento y una tabla de contenido si es largo.
- Usa bloques de código con el lenguaje indicado para los comandos y los fragmentos de Java.
- Incrusta las imágenes del repositorio con rutas relativas, por ejemplo `docs/diagrama-clases.png`.
- Revisa la ortografía y que todos los enlaces funcionen. Abre tu README desde GitHub, en una ventana de incógnito, antes de entregar.

Entregables y estructura del repositorio

Se entrega un solo enlace: el de tu repositorio público de GitHub, con la versión `v1.0.0` publicada. Todo lo demás debe estar dentro.

Entregable	Ubicación
Modelo de análisis (tabla de clases y objetos)	`docs/analisis.md`
Diagrama de clases (imagen y archivo fuente editable)	`docs/diagramas/`
Diagrama de casos de uso (opcional)	`docs/diagramas/`
Código fuente en Java organizado por paquetes	`src/`
README con la documentación	`README.md`
Capturas de la ejecución	`docs/capturas/`
Reglas de ignorado de Git	`.gitignore`

Estructura sugerida

```
petcare-poo-java/
├── README.md
├── .gitignore
├── docs/
│   ├── analisis.md
│   ├── diagramas/
│   └── capturas/
└── src/
    ├── app/
    │   └── Main.java
    └── modelo/
        ├── personas/
        ├── mascotas/
        ├── servicios/
        └── clinica/
```

Puedes cambiar los nombres de los paquetes si justificas la decisión en el README. Lo que no debe cambiar es que el código esté separado por responsabilidades.

Fecha y forma de entrega

La respuesta del presente examen se entregará a través de la plataforma EMINUS (Actividades: Examen Parcial Integrador), Fecha límite de entrega: 2 de nov. 2026. Después de la fecha límite no se revisan commits nuevos.

Rúbrica de evaluación (100 puntos)

Criterio	Puntos	Qué se evalúa
Análisis y diseño UML	20	Tabla de clases y objetos completa, diagrama de clases correcto (notación, visibilidad, multiplicidad), relaciones bien elegidas y justificadas, paquetes representados.
Clases, encapsulamiento y constructores	15	Clases con responsabilidad clara, atributos privados, getters y setters con validación, constructores, sobrecarga y manejo de la creación y eliminación de objetos.
Implementación de relaciones	15	Herencia, clase abstracta, sobreescritura, asociación, agregación y composición implementadas como el diseño indica, con el ciclo de vida correcto.
Paquetes e interfaces	10	Organización por paquetes, importaciones, niveles de acceso con criterio, interfaz definida, implementada y utilizada.
Polimorfismo	10	Uso real del polimorfismo sin instanceof, mediante clases abstractas e interfaces, y explicación de su ventaja.
Funcionamiento y validaciones	5	El programa compila, ejecuta el escenario de demostración y valida entradas y reglas de negocio sin cerrarse.
Convenciones de codificación	5	Nombres, sangría, Javadoc y limpieza del código.
Git y GitHub	10	Historial constante con mensajes claros, ramas, .gitignore, versión v1.0.0.
README profesional	10	Secciones completas, claridad, diagramas y capturas incrustados, instrucciones que funcionan y reflexión personal.

Criterios generales de calificación

- Excelente (90 a 100). Cumple todo y se nota comprensión: las decisiones están justificadas y el código y el modelo coinciden.

- Satisfactorio (70 a 89). Cumple lo esencial, con detalles por mejorar o decisiones poco justificadas.
- Insuficiente (menos de 70). Faltan temas centrales, el código no corresponde al diseño o no se puede ejecutar.

Penalizaciones

- Código que no compila: máximo 60 puntos en total.
- Un solo commit o commits concentrados el último día: se pierde la mitad del puntaje de Git y GitHub.
- Evidencia de trabajo copiado (cero en todo el examen).

Cronograma sugerido para la semana

Es una guía para repartir el trabajo; no tienes que seguirla al pie de la letra, pero sí avanzar cada día para que tu historial de commits lo refleje.

Día	Actividad	Resultado en el repositorio
1	Leer el caso con calma, subrayar sustantivos y verbos, crear el repositorio y el `.gitignore`.	Repositorio creado y primer commit con README inicial.
2	Identificar clases y objetos, y llenar la tabla de análisis.	`docs/analisis.md`
3	Dibujar el diagrama de clases y justificar las relaciones.	Diagrama en `docs/diagramas/`
4	Programar las clases base: `Persona`, `Mascota`, constructores, encapsulamiento y herencia.	Rama `feature/herencia-mascotas` fusionada.
5	Programar interfaces, polimorfismo, citas, agregación y composición.	Rama `feature/citas` fusionada.
6	Construir el menú, validar las reglas de negocio y probar el escenario de demostración.	Programa completo y funcionando.
7	Pulir las convenciones, escribir el README final, tomar capturas y publicar `v1.0.0`	Entrega final.

Integridad académica y uso responsable de la IA

El objetivo de este examen es que construyas conocimiento. Puedes **usar la IA como apoyo**, **pero no como sustituto de tu comprensión**.

Usos que sí se esperan

- Pedir que te explique un concepto que no te quedó claro (por ejemplo, la diferencia entre agregación y composición) y comprobarlo con tus apuntes.
- Pedir ayuda para entender un mensaje de error del compilador.
- Pedir una revisión de ortografía o claridad de tu README, que luego tú editas.
- Pedir ejemplos distintos a tu caso para practicar un tema.

Usos que no son válidos

- Pedir que genere el diseño o el código completo del examen y entregarlo como propio.
- Copiar código que no puedes explicar línea por línea.
- Compartir tu solución con otros compañeros o copiar la de alguien más.

Si usaste IA, indícalo con honestidad en la sección de reflexión del README: para qué la usaste y qué aprendiste de ella. Declararlo no te penaliza; ocultarlo sí.